

Trabajo Práctico: Gestión de Clientes en una Red Social Empresarial

Introducción

Una empresa está desarrollando una red social exclusiva para sus clientes. En esta plataforma, los clientes pueden interactuar siguiendo a otros usuarios, gestionando sus conexiones y realizando acciones sobre su cuenta. El sistema se desarrollará en **tres iteraciones**, cada una con requerimientos específicos que permitirán construir una solución incremental y robusta.

Nota: Se valorará especialmente el uso de **búsquedas eficientes** en las implementaciones. Aquellos estudiantes que logren implementar operaciones con menor complejidad temporal y espacial obtendrán **mayor puntaje**.

Contenido

Introducción	1
Requerimientos Generales del Sistema	2
Formato del Archivo JSON	3
Iteración 1: Gestión Básica de Clientes y Acciones	4
Iteración 2: Relaciones entre Clientes	5
Iteración 3: Relaciones Generales	6
Objetivos del Trabajo	7
Preguntas de Análisis	8
Criterios de Evaluación	9

Requerimientos Generales del Sistema

1. Gestión de Clientes:

- Cada cliente tiene un **nombre** y un **scoring** (un valor que representa su nivel de influencia en la red).
- Se debe permitir la **búsqueda de clientes** por nombre y scoring. **Nota:** Las búsquedas deben ser eficientes (evitar búsquedas secuenciales siempre que sea posible).

2. Historial de Acciones:

- Cada acción realizada en la plataforma debe registrarse para permitir su seguimiento.
- Se debe poder **deshacer la última acción** realizada.

3. Persistencia y Pruebas con Archivo JSON:

- La información inicial de los clientes y sus relaciones deberá cargarse desde un archivo JSON.
 - El sistema deberá ser capaz de leer los datos de este archivo y construir las estructuras de datos correspondientes.
 - Se debe diseñar un conjunto de **pruebas unitarias** para verificar que el sistema funciona correctamente con los datos cargados.
-

Formato del Archivo JSON

El archivo JSON puede tener el siguiente esquema de ejemplo:

```
{
  "clientes": [
    {
      "nombre": "Alice",
      "scoring": 95,
      "siguiendo": ["Bob", "Charlie"],
      "conexiones": ["Bob", "Charlie", "David"]
    },
    {
      "nombre": "Bob",
      "scoring": 88,
      "siguiendo": ["David"],
      "conexiones": ["Alice", "David"]
    },
    {
      "nombre": "Charlie",
      "scoring": 80,
      "siguiendo": [],
      "conexiones": ["Alice", "Eve"]
    },
    {
      "nombre": "David",
      "scoring": 92,
      "siguiendo": ["Alice"],
      "conexiones": ["Bob", "Alice"]
    },
    {
      "nombre": "Eve",
      "scoring": 85,
      "siguiendo": [],
      "conexiones": ["Charlie"]
    }
  ]
}
```

Iteración 1: Gestión Básica de Clientes y Acciones

Fecha de entrega y presentación grupal: 5/2/2026

En esta iteración, el foco estará en la gestión básica de clientes y el historial de acciones.

Agregar el diseño de los tipos abstractos de datos e invariantes de representación.

Requerimientos Específicos:

1. Gestión de Clientes:

- Implementar una estructura para almacenar y gestionar los clientes.
- Permitir la búsqueda de clientes por nombre y scoring. **Nota:** Se valorará el uso de estructuras que permitan búsquedas eficientes (por ejemplo, $O(\log n)$ o mejor).

2. Historial de Acciones:

- El sistema debe mantener un **historial de acciones** realizadas por los clientes en la plataforma, donde cada acción registrada incluya el **tipo de acción** (por ejemplo, "agregar cliente" o "seguir cliente"), los **detalles específicos** (como el nombre del cliente afectado) y, opcionalmente, la **fecha y hora**. El sistema debe permitir **registrar una acción** de manera eficiente, **deshacer la última acción** realizada (eliminandola del historial y devolviendo sus detalles para revertir sus efectos) y, opcionalmente, **consultar todas las acciones** registradas. Las operaciones de registrar y deshacer acciones deben ser **eficientes**, preferiblemente en tiempo constante ($O(1)$).

3. Gestión de Solicitudes de Seguimiento:

- Implementar un mecanismo para procesar solicitudes de seguimiento en el orden en que fueron enviadas.

4. Persistencia y Pruebas:

- Cargar la información inicial de clientes desde el archivo JSON.
 - Implementar pruebas unitarias para validar el correcto funcionamiento del sistema.
-

Iteración 2: Relaciones entre Clientes

Fecha de entrega y presentación grupal: 24/2/2026

En esta iteración, se introducirán las relaciones entre clientes.

Agregar el diseño de los tipos abstractos de datos e invariantes de representación.

Requerimientos Específicos:

1. Relaciones entre Clientes:

- Cada cliente puede seguir hasta **dos clientes**.
- Implementar una estructura para gestionar estas relaciones de manera eficiente.
- Los clientes deben estar ordenados por scoring para facilitar búsquedas. **Nota:** Se valorará el uso de estructuras que permitan búsquedas y operaciones eficientes (por ejemplo, $O(\log n)$).

2. Consultas de Conexiones:

- Permitir la consulta de las conexiones de un cliente (a quiénes sigue).
- Utilizar una implementación de Árbol binario de búsqueda, visto en clase, cargando los datos de los clientes que se siguen e imprimir los clientes que están en el cuarto nivel, para ver quien tiene más seguidores.

3. Persistencia y Pruebas:

- Extender la carga de datos desde el archivo JSON para incluir las relaciones de seguimiento.
 - Implementar pruebas unitarias para validar las nuevas funcionalidades.
-

Iteración 3: Relaciones Generales

Fecha de entrega y presentación grupal: 24/2/2026

En esta iteración, se modelan relaciones más generales entre clientes.

Agregar el diseño de los tipos abstractos de datos e invariantes de representación.

Requerimientos Específicos:

1. Relaciones Generales:

- Implementar una estructura para gestionar relaciones generales entre clientes (amistades, conexiones).
- Permitir agregar relaciones y obtener los vecinos (clientes relacionados) de un cliente. **Nota:** Se valorará el uso de estructuras que permitan operaciones eficientes (por ejemplo, $O(1)$ para obtener vecinos).

2. Distancia entre Clientes:

- Implementar un mecanismo para calcular la distancia (número de saltos) entre dos clientes en la red.

3. Persistencia y Pruebas:

- Extender la carga de datos desde el archivo JSON para incluir las relaciones generales.
 - Implementar pruebas unitarias para validar las nuevas funcionalidades.
-

Objetivos del Trabajo

- Identificar y justificar las estructuras de datos más adecuadas para cada requerimiento.
 - Implementar la solución en Java, asegurando que sea eficiente y bien organizada.
 - Leer la información inicial desde un archivo JSON.
 - Escribir tests unitarios para validar el correcto funcionamiento del sistema.
 - Analizar la complejidad temporal y espacial de las operaciones implementadas.
-

Preguntas de Análisis

1. ¿Qué estructuras de datos son más adecuadas para gestionar clientes, relaciones de seguimiento y relaciones generales? ¿Por qué?
2. ¿Cómo se puede optimizar la búsqueda de clientes por scoring?
3. ¿Cuál es el tipo abstracto de datos para modelar relaciones generales entre clientes?
4. ¿Cómo se puede garantizar que el sistema sea eficiente y escalable con un gran número de clientes y conexiones?

Criterios de Evaluación

1. **Correctitud (40%):**
 - El sistema debe cumplir con todos los requerimientos funcionales.
 - Las pruebas unitarias deben validar el correcto funcionamiento.
2. **Eficiencia (30%):**
 - Se valorará especialmente el uso de estructuras de datos y algoritmos que permitan búsquedas y operaciones eficientes.
 - Las operaciones críticas (búsquedas, consultas, etc.) deben tener una complejidad temporal y espacial óptima.
3. **Organización y Claridad del Código (20%):**
 - El código debe estar bien organizado, modular y documentado.
 - Se debe justificar la elección de las estructuras de datos utilizadas.
4. **Pruebas Unitarias (10%):**
 - Debe incluir un conjunto de pruebas unitarias que cubran los casos principales.

Trabajo Práctico Obligatorio (TPO) grupal, con una calificación grupal y una calificación individual, y la calificación será en base a esta rúbrica:

Criterio	Puntos	Descripción	Niveles de Desempeño
Participación en clase	3 pts	Contribución activa en las discusiones, preguntas y debates en clase.	(3) Excelente: Participó activamente en clase. (2) Bueno: Participó ocasionalmente. (1) Bajo: Participó mínimamente. (0) Nulo: No participó.
Justificación de los TADs utilizados	3 pts	Explicación clara de por qué se eligieron ciertas estructuras de datos.	(3) Excelente: Justificación completa y clara. (2) Buena: Justificación parcial. (1) Débil: Justificación confusa o incompleta. (0) Nula: No pudo justificar.
Respuestas conceptuales ante cambios en el sistema	4 pts	Capacidad de adaptar la solución si se modifica un requerimiento del sistema.	(4) Excelente: Comprende los cambios y adapta la solución correctamente. (3) Bueno: Comprende la mayoría de los cambios y los ajusta bien. (2) Regular: Le cuesta justificar ajustes. (1) Débil: No puede modificar el sistema adecuadamente. (0) Nulo: No responde o no comprende los cambios.

Según la normativa: “De acuerdo con lo estipulado en el Art. 11 de la Resolución Normativa 17/23 Régimen de Evaluación de grado y pregrado, en el caso de las evaluaciones parciales en donde se proponga una instancia de evaluación grupal, se deberá contar con una defensa individual. Se consignará una calificación para cada presentación (nota grupal y defensa individual), que deberá ser superior a 4 (cuatro) puntos. Si en alguna de las dos instancias se obtiene una calificación inferior, se considerará desaprobada la evaluación.”